nvidia.com

What is Kubernetes?

16–20 minutes

---

1. #
2. A
3. B
4. C
5. D
6. E
7. F
8. G
9. H
10. I
11. J
12. K

13. L
14. M
15. N
16. O
17. P
18. Q
19. R
20. S
21. T
22. U
23. V
24. W
25. X
26. Y
27. Z

Kubernetes is an open-source platform for automating container orchestration—the deployment, scaling, and management of containerized applications.

What is Kubernetes?

Kubernetes provides a framework for deploying, managing, scaling, and failover of distributed containers, which are microservices packaged with their dependencies and configurations.

It builds upon more than ten years of experience developing container-management systems at Google, combined with best-of-breed ideas patterns and practices from the community.

Why Kubernetes?

Software containers were popularized with the introduction of Docker in 2013, and now Moby. A container image packages an entire runtime environment, including an application, plus all its dependencies, libraries and other binaries, and configuration files needed to execute the application. Compared to virtual machines, containers have similar resources and isolation benefits, but are more lightweight because containers virtualize the operating system instead of the hardware. Containers are more portable, taking up less space, using far fewer system resources, and are able to be spun up in seconds. Containers also provide efficiency for developers. Instead of waiting for operations to provision machines, DevOps teams can quickly package an application into a container and deploy it easily and consistently across different platforms, whether a laptop, a private data center, a public cloud, or a hybrid environment.

Image reference https://www.docker.com/resources/what-container

Because of their benefits, containers were an immediate hit with developers and quickly became a popular choice for cloud application deployment. The popularity of containers meant that some organizations were soon running thousands of them, creating a need to automate the management. By making containers easier to manage, Kubernetes became popular and also made containers mainstream by further enabling a microservices architecture that promotes fast delivery and scalable orchestration of cloud-native applications.

The Cloud Native Computing Foundation was established as a Linux Foundation project in 2015 to drive adoption of cloud-native technologies. These include containers, service meshes, microservices, immutable infrastructure, and declarative APIs and align developers around a common set of standards. The CNCF serves as the vendor-neutral home for many of the fastest-growing open source projects, including Kubernetes. The group's work has been credited with helping to prevent forks of the Kubernetes code base from emerging. As a result, every major computing platform and cloud provider now supports the same Kubernetes code base. While branded versions of Kubernetes have emerged, such as Red Hat OpenShift or Amazon Elastic Kubernetes Service, the underlying code is the same.

How Does Kubernetes Work?

Kubernetes introduced a grouping concept called a "pod" that enables multiple containers to run on a host machine and share resources without the risk of conflict. A pod can be used to define shared services—like a directory, IP address or storage—and expose it to all the containers in the pod. This enables services within an application to be containerized and run together because each container is tied tightly to the main application.

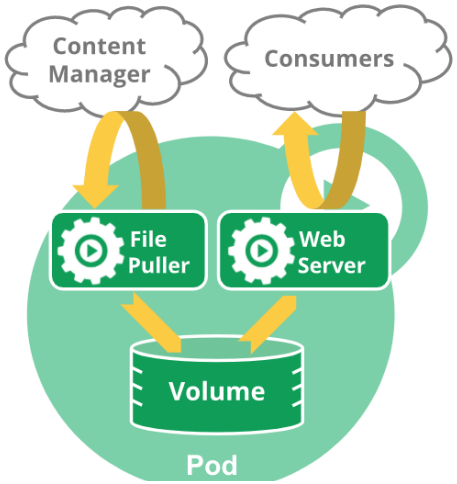


Image reference https://kubernetes.io/docs/concepts/workloads/pods/

A node agent, called a kubelet, manages the pods, containers, and images. Kubernetes controllers manage clusters of pods and ensure that adequate resources are allocated to achieve desired scalability and performance levels.

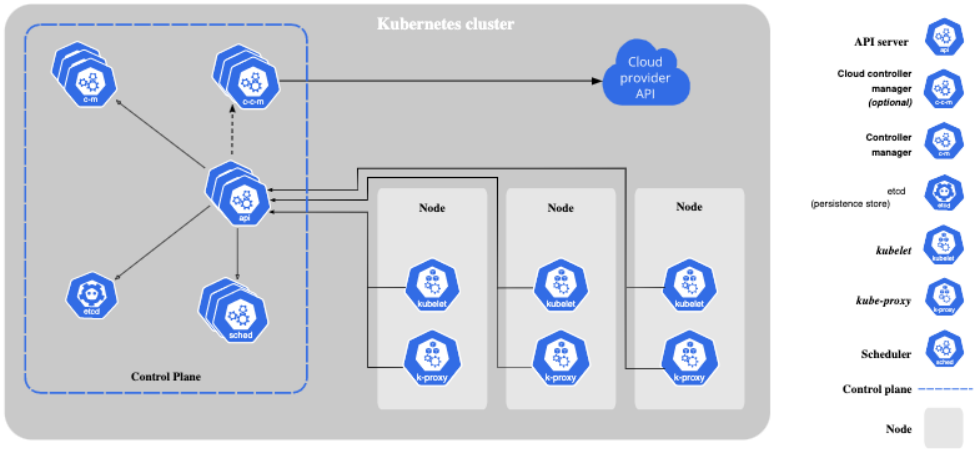


Image reference https://kubernetes.io/docs/concepts/overview/components/

Kubernetes provides a variety of useful services, particularly in clustered environments. It automates service discovery and load balancing, automatically mounts storage systems, and automates rollouts and rollbacks to achieve a specified desired state. It also monitors container health, restarts containers that fail, and enables sensitive information such as passwords and encryption keys to be safely stored in containers.

This simplifies the management of machines and services, enabling a single administrator to manage thousands of containers running simultaneously. Kubernetes also allows orchestration across on-site deployments to public or private clouds and to hybrid deployments in-between.

Kubernetes has generated a lot of excitement about its promise as a platform to enable hybrid cloud

computing. Because the code base is the same in every physical and virtual environment, applications that are containerized can theoretically be run on any platform that supports Kubernetes. The industry continues to debate the merits of hybrid architectures. Advocates say the approach prevents lock-in while detractors argue that the trade-off for portability is that developers are limited to a narrow range of open-source technologies and can't take advantage of the full functionality of services on branded cloud and on-premises platforms.

Use Cases for Kubernetes

As noted above, hybrid and multi-cloud deployments are an excellent use case for Kubernetes because applications need not be tied to an underlying platform. Kubernetes handles resource allocation and monitors container health to ensure that services are available as needed.

Kubernetes is also well-suited for environments in which availability is critical because the orchestrator protects against such problems as failed instances, port conflicts, and resource bottlenecks.

Containers are a foundational technology for serverless computing in which applications are built from services that come alive and execute a function solely for the needs of that application. Serverless computing is a bit of a misnomer as containers must run on a server. But the objective is to minimize the cost and time needed to provision virtual machines by encapsulating them in containers which can be spun up in milliseconds and managed by Kubernetes.

Kubernetes also has a feature called a namespace, which is a virtual cluster within a cluster. This enables operations and development teams to share the same set of physical machines and access the same services without creating conflicts.

Why Kubernetes Matters to…

Data scientists

One of the challenges of data science is to create repeatable experiments in reproducible environments with the ability to track and monitor metrics in production. Containers offer the ability to create repeatable pipelines with multiple coordinated stages that work together in a reproducible way for processing, feature extraction, and testing.

Declarative configurations in Kubernetes describe connections between services. Microservice architectures enable easier debugging and improve collaboration between members of a data science team. Data scientists can also take advantage of extensions like BinderHub, which lets them build and register container images from a repository and publish them as a shared notebook that other users can interact with.

Other extensions like Kubeflow streamline the process of setting up and maintaining machine learning workflows and pipelines in Kubernetes. The portability benefits of the orchestrator make it possible for a data scientist to develop on a laptop computer and deploy anywhere.

Devops

Putting machine learning models into production can be a struggle for data engineers. They spend time editing configuration files, allocating server resources, and worrying about how to scale models and incorporate GPUs without causing the project to crash. The container ecosystem has introduced many tools that are intended to make the data engineer's life easier.

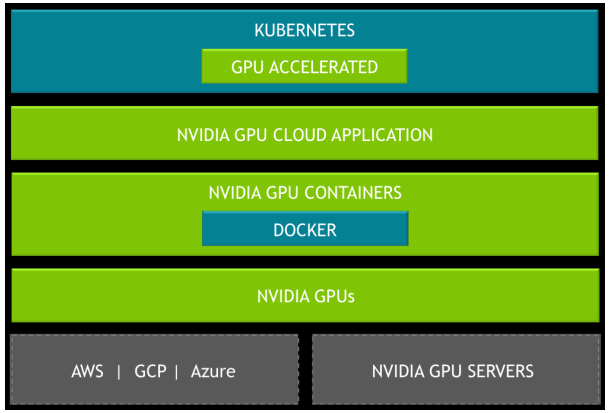
For example, Istio is a configurable, open-source service-mesh layer that makes it easy to create a network of deployed services with automated load balancing, service-to-service authentication, and monitoring with little or no change to service code. It provides fine-grained control of traffic behavior, rich routing rules, retries, failovers, and fault injection, along with a pluggable policy layer and configuration API for access controls, rate limits, and quotas.

The Kubernetes ecosystem is continuing to evolve with such specialized tools to make server configuration invisible and to enable data engineers to visualize dependencies that make configuration and troubleshooting easier.

Why Kubernetes Runs Better on GPUs

Kubernetes includes support for GPUs, making it easy to configure and use GPU resources for accelerating workloads such as data science, machine learning, and deep learning. Device plug-ins enable pods access to specialized hardware features such as GPUs and expose them as schedulable resources.

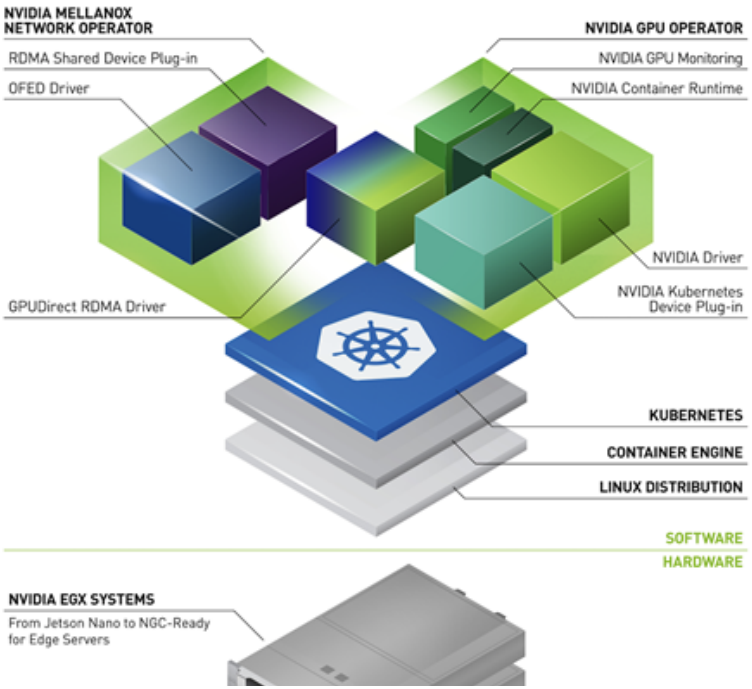
With the increasing number of AI-powered applications and services and the broad availability of GPUs in public cloud, there's an increasing need for Kubernetes to be GPU-aware. NVIDIA has been steadily building its library of software to optimize GPUs to use in a container environment. For example, Kubernetes on NVIDIA GPUs enables multi-cloud GPU clusters to be scaled seamlessly with automated deployment, maintenance, scheduling, and operation of GPU-accelerated containers across multi-node clusters.

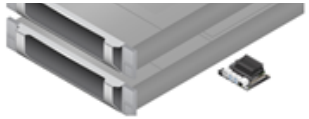


Kubernetes on NVIDIA GPUs has the following key features:

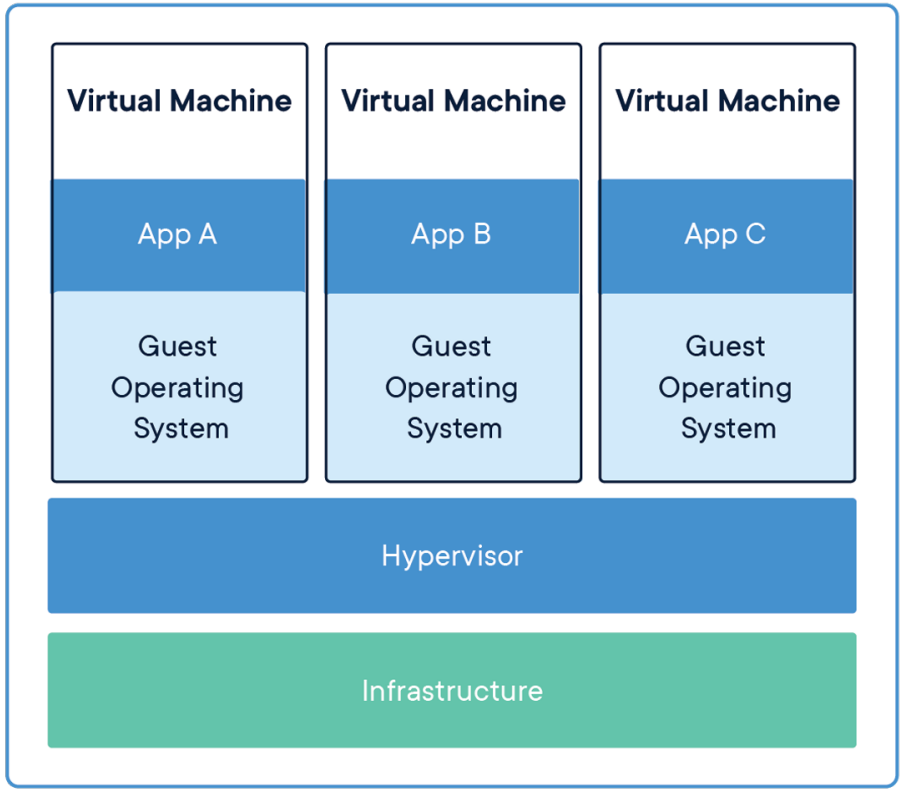
- Enables GPU support in Kubernetes using the NVIDIA device plug-in
- Specifies GPU attributes such as GPU type and memory requirements for deployment in heterogeneous GPU clusters
- Allows visualization and monitoring GPU metrics and health with an integrated GPU monitoring stack of NVIDIA DCGM, Prometheus, and Grafana
- Supports multiple underlying container runtimes such as Docker and CRI-O
- Officially supported on NVIDIA DGX™ systems

The NVIDIA EGX™ stack is a cloud-native and scalable software stack that enables containerized accelerated AI computing managed by Kubernetes. With the NVIDIA EGX stack, organizations can deploy updated AI containers effortlessly in minutes.





Kubernetes isn't, however, a silver bullet. It provides good APIs for resource discovery and management, but it isn't the whole solution when it comes to making those resources easy to use. Because of this, NVIDIA developed Triton, an open-source inference serving platform that enables users to deploy AI training models on any GPU or CPU-based interface. By running Triton in a Kubernetes environment, it's possible to completely abstract the hardware from the software. In this case, Kubernetes is acting as the substrate on which Triton runs. Triton handles the abstraction of the hardware within the node, while Kubernetes orchestrates the cluster, enabling it to scale out more effectively.



Kubernetes in GPU Hardware

Looking beyond software, NVIDIA has taken steps to tailor its hardware for use in virtualized environments as well. This isn't specific to Kubernetes, though. With the launch of the company's Ampere™-based A100 enterprise GPUs and the DGX A100 server, NVIDIA introduced multiple instance GPU (MIG). MIG enables a single A100 GPU to be segmented into seven smaller GPUs, similar to how a CPU can be segmented into its individual cores. This enables users to automatically scale their applications using a container runtime like Kubernetes with much greater granularity.

Prior to MIG, each node in a GPU-accelerated Kubernetes cluster would require its own dedicated GPU. With MIG, a single NVIDIA A100, of which there are eight in a DGX A100, can now support up to seven smaller nodes. This allows the application and resources to scale more linearly.

As AI serving becomes a GPU-accelerated workload, which is just at the inflection point of taking off, GPUs will begin to move into the mainstream of Kubernetes. As things expand, people are going to be able to just think about GPU-accelerated as a fast button or an efficient button and not have to think about GPU development or programming.